# Ejercicio · UD02 · Bloque 2

## Diseña la interacción de un asistente multimodal

| | |
|---|---|
| **Asignatura** | Desarrollo de Interfaces · 2.º DAM |
| **Unidad** | UD02 · Generación de interfaces naturales de usuario |
| **Contenido trabajado** | Diapositivas 19–26 · Bloque 2: Interfaces naturales, tipos (PDF, apartado 2, pp. 14–18) |
| **Temas** | Canales y tipos de NUI · entrada explícita frente a interpretada · estados (NaturalUiState) · evolución de la interacción · eventos GUI/NUI en JavaFX · tipos de interfaz y cuándo usarlos · campos de aplicación · política de interacción (InteractionPolicy) |
| **Modalidad** | Individual |

## Contexto

Un taller mecánico quiere una aplicación para tableta llamada TallerGuía. La tableta va sujeta a un soporte junto al vehículo y el mecánico la usa mientras trabaja, normalmente con guantes y las manos sucias. El taller es ruidoso (compresores, herramientas neumáticas, radio).

La aplicación debe permitir:

- Avanzar, repetir o volver al paso anterior de una reparación guiada.
- Señalar sobre la imagen de la cámara la pieza a la que se refiere el paso (realidad aumentada).
- Pedir un recambio al almacén (genera un pedido con coste para el taller).
- Cerrar la orden de trabajo (el vehículo pasa a «listo para entregar» y se avisa al cliente).
- Consultar el manual técnico del vehículo.

Las intenciones internas que usa la aplicación son: NEXT_STEP, REPEAT_STEP, SHOW_MANUAL, MARK_PART, ORDER_PART y CLOSE_WORK_ORDER.

## Ejercicio

### a) Canales de interacción (diap. 19 y 25)

- Indica qué canales de interacción combinarías en TallerGuía y cuál sería el canal dominante. ¿Qué tipo de NUI es según su señal?

#### Respuesta a.1 · Combinación de canales, canal dominante y tipo de NUI

En TallerGuía combinaría cuatro canales: voz (entrada), táctil (entrada explícita y respaldo), cámara/visión (entrada de contexto y salida en realidad aumentada) y pantalla + audio (salida: texto grande, resaltado AR y tonos de aviso).

**Canal dominante:** la voz. El mecánico trabaja con las manos ocupadas, con guantes y sucias, así que el canal que no exige tocar nada es el más natural para las órdenes frecuentes y cortas (diap. 19: el canal se elige según el contexto de uso del usuario). Para que el ruido de compresores y herramientas no la haga poco fiable, la voz se activa con el botón del micrófono (pulsar para hablar) y no con escucha permanente, y se limita a un vocabulario corto y cerrado («siguiente», «repite», «manual», «marca la pieza»).

**Tipo de NUI según su señal:** es una NUI multimodal cuya señal principal es acústica (de voz), complementada con señal visual (la cámara y la superposición AR) y señal táctil (botones grandes). No elijo gestos como canal: con guantes, suciedad y una tableta fija junto al vehículo, la cámara no tendría un encuadre estable del mecánico y los gestos darían falsos positivos mientras trabaja.

- Asigna un canal a cada una de las cinco funciones de la lista y justifica la elección. Ten en cuenta las condiciones del taller.

### Respuesta a.2 · Canal asignado a cada función

| Función | Canal elegido | Justificación (condiciones del taller) |
|---|---|---|
| Avanzar, repetir o volver al paso anterior | Voz (dominante) + botones táctiles grandes de respaldo | Son órdenes muy frecuentes, de una o dos palabras, que se dicen con las manos ocupadas. Son reversibles y sin coste, así que un fallo de reconocimiento es poco grave. Si el ruido impide la voz, el botón grande sigue siendo accesible con guante. |
| Señalar la pieza en la imagen de la cámara (AR) | Visual: cámara + superposición AR (se dispara por voz o al cambiar de paso) | Dónde está una pieza es información espacial: es mucho más rápido verla resaltada sobre la imagen real que describirla con palabras o texto. No depende del ruido ni de las manos. |
| Pedir un recambio al almacén | Voz para iniciar («pide un filtro de aceite») + táctil para confirmar | Decirlo es cómodo, pero genera un coste para el taller y el ruido puede hacer que se entienda otra pieza. La confirmación debe ser explícita y visual (pieza, cantidad y coste en pantalla) y con un botón grande. |
| Cerrar la orden de trabajo | Táctil (confirmación explícita, p. ej. botón grande o mantener pulsado) | Cambia el estado del vehículo a «listo para entregar» y avisa al cliente: efecto externo difícil de deshacer. Debe ser un acto deliberado, no algo que una palabra mal oída pueda disparar. |
| Consultar el manual técnico | Voz para abrir/buscar el apartado + pantalla táctil para leer y desplazarse | La salida es necesariamente visual (esquemas, tablas, pares de apriete). Dictar una búsqueda larga en un taller ruidoso es poco fiable, por eso se combina con zoom y desplazamiento táctil con botones grandes. |

- Según su campo de aplicación, ¿qué requisitos de precisión, trazabilidad y validación debería cumplir? Compáralo con una app de ocio.

### Respuesta a.3 · Campo de aplicación: precisión, trazabilidad y validación

TallerGuía pertenece al campo profesional/industrial (diap. 25): hay dinero, seguridad y obligaciones con un cliente en juego. Por eso debe cumplir requisitos mucho más estrictos que una app de ocio:

| Requisito | TallerGuía (profesional/industrial) | App de ocio (p. ej. juego o reproductor de música) |
|---|---|---|
| Precisión | Alta. Hay que identificar bien la pieza, la referencia y el paso. Una orden mal interpretada cuesta dinero o puede llevar a trabajar sobre el elemento equivocado. Se usan umbrales de confianza del reconocimiento y vocabulario cerrado. | Media-baja. Un fallo es una molestia: se repite la orden o se ignora. |
| Trazabilidad | Alta. Debe quedar registro de quién dio la orden, cuándo, sobre qué orden de trabajo y vehículo, qué se entendió y qué se confirmó (pedidos y cierres). Sirve para auditar, resolver disputas con el almacén o el cliente y detectar errores. | Mínima o nula. Como mucho estadísticas de uso anónimas. |
| Validación | Alta. Confirmación explícita antes de acciones con coste o efecto externo; comprobar que la pieza existe y es compatible con el vehículo y que los pasos están completos antes de cerrar. | Baja. Se ejecuta directamente; deshacer es sencillo y no hay coste ni consecuencias externas. |
| Prioridad de diseño | Fiabilidad y seguridad por encima de la espectacularidad. | Fluidez, diversión y naturalidad por encima de la exactitud. |

## b) Entrada interpretada y estados (diap. 20 y 21)

El mecánico pulsa el botón del micrófono y dice: «pide un filtro de aceite».

- Escribe la secuencia de estados de NaturalUiState por la que pasa la interfaz hasta que el pedido se realiza, y describe qué ve el mecánico en la pantalla en cada estado.

### Respuesta b.1 · Secuencia de estados de NaturalUiState

Orden dicha: «pide un filtro de aceite» (intención ORDER_PART, que tiene coste y por eso pasa por confirmación). Secuencia: IDLE → LISTENING → PROCESSING → CONFIRMING → EXECUTING → SUCCESS → IDLE.

| # | Estado | Qué provoca el cambio | Qué ve el mecánico en pantalla |
|---|---|---|---|
| 1 | IDLE | Estado inicial. | Paso actual de la reparación, botón grande del micrófono activo, mensaje «Sistema en espera». «Cancelar» deshabilitado. |
| 2 | LISTENING | Pulsa el botón del micrófono. | Botón del micrófono deshabilitado y animado, indicador de nivel de sonido y texto «Escuchando…». «Cancelar» habilitado. |
| 3 | PROCESSING | Termina de hablar (o salta el silencio). | Transcripción «pide un filtro de aceite», indicador de progreso y «Interpretando…» mientras se obtiene la intención ORDER_PART y la pieza. |
| 4 | CONFIRMING | La intención tiene coste: la política exige confirmación. | Tarjeta con la pieza, la referencia, la cantidad y el coste; botones grandes «Confirmar pedido» y «Cancelar». No se pide nada todavía. |
| 5 | EXECUTING | Toca «Confirmar pedido». | «Enviando pedido al almacén…» con indicador de progreso; los botones de la tarjeta quedan bloqueados para evitar un doble pedido. |
| 6 | SUCCESS | El almacén acepta el pedido. | Marca verde y «Pedido enviado: filtro de aceite», con tono de aviso. |
| 7 | IDLE | Pasados unos segundos, vuelve solo a reposo. | Vuelve al paso de la reparación en el que estaba; el pedido queda registrado. |

- Indica dos cosas que pueden fallar en la interpretación de esta orden en el taller y en qué estado acabaría la interfaz en cada caso.

### Respuesta b.2 · Dos fallos posibles en la interpretación

- **Fallo 1.** El ruido tapa la voz. Con un compresor o una llave neumática de fondo, la transcripción sale ininteligible («pide un filtro de ahí te») o no sale nada. No se obtiene ninguna intención reconocible o la confianza es baja. La interfaz pasa de LISTENING o PROCESSING a ERROR, con el mensaje «No te he entendido» y la opción de reintentar.
- **Fallo 2.** Se entiende otra pieza o es ambigua. El reconocedor oye «filtro de aire» en lugar de «filtro de aceite», o «filtro» coincide con varias referencias del catálogo para ese vehículo. Si la orden parece válida pero la pieza es errónea, la interfaz acaba en CONFIRMING mostrando la pieza equivocada: el mecánico lo ve y pulsa «Cancelar» (→ IDLE) o repite la orden. Esa pantalla de confirmación es precisamente lo que evita un pedido erróneo con coste. Si el catálogo no devuelve ninguna pieza compatible, la interfaz acaba en ERROR con «No encuentro ese recambio para este vehículo».
- Explica cómo sale la interfaz del estado ERROR sin bloquear al mecánico.

### Respuesta b.3 · Cómo se sale de ERROR sin bloquear al mecánico

El estado ERROR nunca debe ser un callejón sin salida, y menos con guantes y las manos ocupadas. Por eso:

- Muestra un mensaje corto y claro, sin diálogos modales con botones diminutos, y mantiene visible el paso de la reparación en curso (el error no hace perder el contexto).
- Ofrece dos botones grandes: «Reintentar» (→ LISTENING) y «Cancelar» (→ IDLE). Cancelar es la misma acción que en cualquier otro estado distinto de IDLE.
- Si nadie toca nada, vuelve solo a IDLE tras unos segundos (temporizador), para que el sistema nunca quede atascado.
- Tras dos o tres fallos seguidos propone el canal alternativo táctil (buscar la pieza en el catálogo de la pantalla), porque la voz no está funcionando en ese ruido.
- Un error no ejecuta ninguna acción: no se hace ningún pedido y la orden de trabajo no cambia.

## c) ¿NUI o interfaz tradicional? (diap. 22 y 24)

Para cada tarea, indica el tipo de interfaz más adecuado (tradicional, voz, gestual/corporal o inmersiva AR/VR) y justifícalo con el criterio «cuándo usarla»:

| Tarea | Tipo de interfaz | Justificación |
|---|---|---|
| 1. Avanzar al siguiente paso de la reparación | Voz (NUI), con botón táctil grande de respaldo | Se usa la voz cuando las manos están ocupadas o se quiere interactuar sin tocar el dispositivo, y la orden es corta y de vocabulario cerrado (diap. 24). «Siguiente» es una orden de una palabra, reversible (se puede repetir o volver) y sin coste, así que un fallo ocasional del reconocimiento es poco grave. |
| 2. Ver dónde está el sensor dentro del motor | Inmersiva AR (realidad aumentada sobre la imagen de la cámara) | La AR se usa cuando la información está ligada a un objeto o lugar físico y se entiende mejor superpuesta a la realidad (diap. 24). Señalar el sensor dentro del motor es un dato espacial: verlo resaltado sobre la imagen real es más rápido y menos ambiguo que leer una descripción. Es AR (en la tableta), no VR, porque el mecánico tiene que seguir viendo el motor real. |
| 3. Emitir la factura al cliente | Tradicional (GUI: formulario, teclado y ratón/táctil) | La interfaz tradicional es la mejor cuando hay datos precisos que revisar y validar y no hay urgencia de manos libres. La factura lleva importes, impuestos y datos fiscales del cliente que deben introducirse y comprobarse con exactitud, y tiene validez legal. Se hace en la oficina, no en el foso. |
| 4. Corregir el stock de 40 referencias en el inventario | Tradicional (GUI: tabla editable con teclado, lector de códigos o importación) | Es una tarea repetitiva, masiva y numérica: 40 referencias con su cantidad. Con una tabla editable, el tabulador y el teclado numérico se hace rápido y se ve todo de una vez para revisarlo. Dictar 40 cifras en un taller ruidoso sería lento, propenso a errores y sin forma cómoda de comprobarlos (diap. 22 y 24). |

### Respuesta c · «La complejidad no desaparece, se traslada» (diap. 22)

Quiere decir que una interfaz natural no elimina la complejidad de la tarea: la quita del usuario y la pone en el sistema (y en quien lo diseña). En TallerGuía:

- **Antes (GUI):** el mecánico traduce lo que quiere hacer a una secuencia de pasos de la interfaz: Pedidos → Nuevo pedido → buscar «filtro de aceite» → elegir referencia → cantidad → confirmar. Es un esfuerzo físico y mental suyo, pero cada acción es explícita y sin ambigüedad.
- **Ahora (NUI):** solo dice «pide un filtro de aceite». El esfuerzo desaparece para él, pero la aplicación tiene que resolver lo que antes resolvía el usuario: reconocer la voz en medio del ruido, interpretar la intención (ORDER_PART) y la pieza entre sinónimos, decidir cuándo preguntar («¿filtro de aceite del motor o de la caja de cambios?»), gestionar los estados (NaturalUiState), pedir confirmación cuando hay coste y recuperarse de errores (estado ERROR).
- También se traslada al diseñador/programador: más estados, más casos límite, más pruebas en entornos reales (ruido, guantes, luz) y una política de interacción que decida qué se puede ejecutar sin confirmar. Y parte vuelve al usuario de otra forma: tiene que aprender qué puede decir y fijarse en el estado de la pantalla para saber si el sistema le ha entendido.

## d) Eventos en JavaFX (diap. 23)

- Escribe tres eventos de tipo NUI que generaría TallerGuía y otros tres de tipo GUI clásica.

### Respuesta d.1 · Eventos NUI y eventos GUI de TallerGuía

| Tipo | Evento | Qué provoca |
|---|---|---|
| NUI | Comando de voz reconocido («siguiente», «repite», «pide un filtro de aceite») | El servicio de reconocimiento entrega el texto y la intención (NEXT_STEP, REPEAT_STEP, ORDER_PART…); la interfaz pasa a PROCESSING. |
| NUI | Pieza detectada en la imagen de la cámara (AR) | La visión localiza la pieza del paso y la aplicación dibuja el resaltado sobre ella (intención MARK_PART). |
| NUI | Reconocimiento fallido o de baja confianza (ruido, silencio, tiempo de espera agotado) | La interfaz pasa a ERROR y ofrece reintentar o cancelar. |
| GUI | Clic/toque en el botón «Activar modo voz» (ActionEvent) | Pasa a LISTENING e inicia la captura de voz. |
| GUI | Clic/toque en «Confirmar pedido» (ActionEvent) | Confirma la acción con coste y pasa a EXECUTING. |
| GUI | Desplazamiento/zoom en el manual técnico (ScrollEvent / ZoomEvent) | Mueve o amplía el contenido del manual. |

**Diferencia clave:** los eventos GUI los genera JavaFX directamente (setOnAction, ScrollEvent…). Los NUI los genera un servicio externo (voz, visión) que corre en otro hilo y llama de vuelta a la aplicación; por eso, al actualizar la pantalla desde ese servicio hay que usar Platform.runLater(...).

- La pantalla guarde su estado con NaturalUiState:
  - un atributo estado que empieza en IDLE;
  - un método cambiarEstado(NaturalUiState nuevo, String texto) que actualiza el atributo y la etiqueta;
  - el botón «Activar modo voz» pasa a LISTENING y queda deshabilitado mientras escucha;
  - un botón «Cancelar» que vuelve a IDLE y solo está habilitado cuando no se está en reposo.

```
Button voiceButton = new Button("Activar modo voz");
Label statusLabel = new Label("Sistema en espera");

voiceButton.setOnAction(event -> {
    statusLabel.setText("Escuchando comando del usuario...");
    // Aquí se conectaría el servicio de reconocimiento de voz.
});
```

### Respuesta d.2 · Pantalla con estado (archivos NaturalUiState.java y TallerGuiaPantalla.java)

Amplío el ejemplo de la diap. 23 con un atributo estado (empieza en IDLE) y un método cambiarEstado(nuevo, texto) que actualiza el atributo y la etiqueta y, de paso, mantiene coherentes los botones: «Activar modo voz» queda deshabilitado solo en LISTENING y «Cancelar» solo está habilitado cuando el estado no es IDLE. Si se centraliza todo en cambiarEstado, ningún evento puede dejar la pantalla en un estado incoherente.

NaturalUiState.java (los nombres de los estados están basados en los que describe la unidad; si el enum de clase tiene otros, solo hay que adaptar esta lista):

```
/**
 * Estados por los que pasa la interfaz natural de TallerGuía (UD02, diap. 21).
 *
 * NOTA: si el enum de la unidad usa otros nombres, basta con adaptar esta
 * lista; el resto del código solo depende de IDLE, LISTENING y de que
 * existan los demás valores.
 */
public enum NaturalUiState {
    IDLE,        // reposo: el sistema espera una orden
    LISTENING,   // captando la voz del mecánico
    PROCESSING,  // interpretando la orden (reconocimiento + intención)
    CONFIRMING,  // pide confirmación explícita antes de una acción con coste
    EXECUTING,   // ejecutando la acción confirmada (p. ej. enviando el pedido)
```

```
    SUCCESS,     // la acción se ha ejecutado y se informa del resultado
    ERROR        // no se entendió / falló algo; se ofrece salida inmediata
}
```

**TallerGuiaPantalla.java:**

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

/**
 * Apartado d) · Pantalla de TallerGuía que guarda su estado con NaturalUiState.
 * Amplía el ejemplo de la diapositiva 23 (botón de voz + etiqueta de estado).
 */
public class TallerGuiaPantalla extends Application {

    // Atributo estado: la pantalla empieza en reposo.
    private NaturalUiState estado = NaturalUiState.IDLE;

    private final Button voiceButton = new Button("Activar modo voz");
    private final Button cancelButton = new Button("Cancelar");
    private final Label statusLabel = new Label("Sistema en espera");

    /** Actualiza el atributo estado y la etiqueta, y deja los botones coherentes. */
    private void cambiarEstado(NaturalUiState nuevo, String texto) {
        estado = nuevo;
        statusLabel.setText(texto);
        // «Activar modo voz» queda deshabilitado mientras se escucha.
        voiceButton.setDisable(estado == NaturalUiState.LISTENING);
        // «Cancelar» solo está habilitado cuando NO se está en reposo.
        cancelButton.setDisable(estado == NaturalUiState.IDLE);
    }

    @Override
    public void start(Stage stage) {
        // Evento GUI/NUI del ejemplo de la diapositiva 23, ahora con estado.
        voiceButton.setOnAction(event -> {
            cambiarEstado(NaturalUiState.LISTENING, "Escuchando comando del usuario...");
            // Aquí se conectaría el servicio de reconocimiento de voz.
        });

        // Cancelar: vuelve siempre a reposo.
        cancelButton.setOnAction(event ->
                cambiarEstado(NaturalUiState.IDLE, "Sistema en espera"));

        // Coherencia inicial (IDLE => Cancelar deshabilitado).
        cambiarEstado(NaturalUiState.IDLE, "Sistema en espera");

        VBox root = new VBox(12, statusLabel, voiceButton, cancelButton);
        root.setStyle("-fx-padding: 24;");
        stage.setTitle("TallerGuía");
        stage.setScene(new Scene(root, 360, 200));
        stage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Comportamiento: al arrancar se llama a cambiarEstado(IDLE, …) y «Cancelar» queda deshabilitado. Al pulsar «Activar modo voz» → LISTENING: la etiqueta pasa a «Escuchando comando del usuario…», el botón de voz se deshabilita (no se puede iniciar otra escucha encima) y «Cancelar» se habilita. Al pulsar «Cancelar» → IDLE: se restablece «Sistema en espera», el botón de voz se vuelve a habilitar y «Cancelar» se deshabilita. Para ejecutarlo hace falta JavaFX (por ejemplo con --module-path y --add-modules javafx.controls).

## e) Política de interacción (diap. 26)

Partimos de la clase InteractionPolicy de la unidad:

```
public class InteractionPolicy {
    public boolean requiresConfirmation(String intent) {
        return intent.equals("DELETE_DATA") || intent.equals("MAKE_PAYMENT");
    }

    public boolean canRunHandsFree(String intent) {
        return !requiresConfirmation(intent);
    }
}
```

- Completa la tabla sin ejecutar el código (si una llamada provoca un error, indícalo):

| intent | requiresConfirmation | canRunHandsFree |
|---|---|---|
| "NEXT_STEP" | false | true |
| "MAKE_PAYMENT" | true | false |
| "ORDER_PART" | false | true |
| "UNKNOWN" | false | true |
| "delete_data" | false | true |
| null | NullPointerException | NullPointerException |

### Respuesta e.1 · Razonamiento de la tabla

- **"NEXT_STEP"** → no es DELETE_DATA ni MAKE_PAYMENT, así que requiresConfirmation = false y canRunHandsFree = !false = true.
- **"MAKE_PAYMENT"** → coincide con una de las dos cadenas: requiresConfirmation = true y canRunHandsFree = false.
- **"ORDER_PART"** → no está en la lista, así que se trata como segura: false / true (aunque en TallerGuía tiene coste).
- **"UNKNOWN"** → tampoco está en la lista: false / true. Una intención no reconocida se ejecutaría sin confirmar.
- **"delete_data"** → equals distingue mayúsculas de minúsculas, así que no coincide con "DELETE_DATA": false / true (se salta la confirmación de borrado).
- **null** → intent.equals(...) se invoca sobre null y lanza **NullPointerException**. Lo mismo ocurre en canRunHandsFree, que llama a requiresConfirmation.

- A partir de la tabla, identifica al menos tres problemas que tendría usar esta clase tal cual en TallerGuía.

### Respuesta e.2 · Problemas de usar InteractionPolicy tal cual en TallerGuía

- **No conoce las intenciones de TallerGuía.** Solo contempla DELETE_DATA y MAKE_PAYMENT, que no existen en la app. ORDER_PART (genera un pedido con coste) y CLOSE_WORK_ORDER (cambia el estado del vehículo y avisa al cliente) se ejecutarían sin confirmación y con manos libres; en un taller ruidoso, una orden mal oída llegaría directamente al almacén.
- **Falla de forma insegura (lista negra).** Todo lo que no esté en la lista de peligrosas se permite, así que «UNKNOWN» o cualquier intención mal interpretada se ejecuta sin confirmar. Lo seguro es lo contrario: lista blanca y, ante la duda, confirmar.
- **Distingue mayúsculas y minúsculas.** «delete_data» se salta la comprobación. Si el reconocedor o un componente devuelven la intención en otro formato, la política deja de proteger.
- **Se rompe con null** (NullPointerException). Un fallo de reconocimiento que devuelva null tumbaría la aplicación en lugar de llevar al estado ERROR, con el mecánico a mitad de reparación.

- **Cadenas escritas a mano y duplicadas.** Una errata no da error de compilación. Además, canRunHandsFree es solo la negación de requiresConfirmation, y son dos preguntas distintas: «¿necesita confirmación?» no equivale a «¿puede ejecutarse con manos libres?» (una intención desconocida no debería ser ninguna de las dos cosas).
- Escribe una clase TallerPolicy que resuelva esos problemas. ¿Qué intenciones de TallerGuía deben pedir confirmación y cuáles pueden ejecutarse con manos libres? Justifícalo.

### Respuesta e.3 · Clase TallerPolicy (archivo TallerPolicy.java)

Soluciona los problemas anteriores: usa las intenciones reales, **lista blanca** (lo desconocido pide confirmación y no va por manos libres), tolera null/vacío, normaliza mayúsculas y espacios, y separa las dos preguntas.

**TallerPolicy.java:**

```
import java.util.Locale;
import java.util.Set;

/**
 * Apartado e) · Política de interacción de TallerGuía.
 *
 * Diferencias respecto a InteractionPolicy:
 *  1. Usa las intenciones REALES de TallerGuía (no DELETE_DATA / MAKE_PAYMENT).
 *  2. Lista BLANCA: solo lo conocido y seguro se ejecuta sin confirmar;
 *     cualquier intención desconocida exige confirmación (falla de forma segura).
 *  3. Null-safe: null o vacío no lanza NullPointerException; se trata como desconocido.
 *  4. Normaliza mayúsculas/minúsculas y espacios ("order_part " == "ORDER_PART").
 *  5. canRunHandsFree NO es simplemente !requiresConfirmation: una intención
 *     desconocida no puede ejecutarse con manos libres ni tampoco "se confirma sola".
 */
public class TallerPolicy {

    /** Acciones con coste económico o efecto irreversible/externo: confirmación explícita.
*/
    private static final Set<String> REQUIRE_CONFIRMATION =
            Set.of("ORDER_PART", "CLOSE_WORK_ORDER");

    /** Acciones reversibles, sin coste y sin efectos externos: manos libres. */
    private static final Set<String> HANDS_FREE =
            Set.of("NEXT_STEP", "REPEAT_STEP", "SHOW_MANUAL", "MARK_PART",
                   "PREVIOUS_STEP"); // PREVIOUS_STEP: ampliación, ver la nota del documento

    private String normalize(String intent) {
        return intent == null ? "" : intent.trim().toUpperCase(Locale.ROOT);
    }

    /** true si la intención es una de las seis (o PREVIOUS_STEP) que TallerGuía conoce. */
    public boolean isKnown(String intent) {
        String i = normalize(intent);
        return REQUIRE_CONFIRMATION.contains(i) || HANDS_FREE.contains(i);
    }

    /** Pide confirmación si tiene coste/efecto externo, o si no se reconoce la intención.
*/
    public boolean requiresConfirmation(String intent) {
        String i = normalize(intent);
        return REQUIRE_CONFIRMATION.contains(i) || !isKnown(i);
    }

    /** Manos libres solo para intenciones conocidas, seguras y reversibles. */
    public boolean canRunHandsFree(String intent) {
        return HANDS_FREE.contains(normalize(intent));
    }
}
```

**InteractionPolicy.java** (la clase original de la unidad, sin modificar, para poder compararla):

```
/** Clase original de la unidad (diap. 26), sin modificar. */
public class InteractionPolicy {

    public boolean requiresConfirmation(String intent) {
        return intent.equals("DELETE_DATA") || intent.equals("MAKE_PAYMENT");
```

```
    }

    public boolean canRunHandsFree(String intent) {
        return !requiresConfirmation(intent);
    }
}
```

**TallerPolicyDemo.java** (programa que imprime la tabla comparando las dos políticas):

```
/** Comprueba InteractionPolicy (original) frente a TallerPolicy con la tabla del ejercicio.
*/
public class TallerPolicyDemo {

    public static void main(String[] args) {
        String[] intents = { "NEXT_STEP", "MAKE_PAYMENT", "ORDER_PART", "UNKNOWN",
                             "delete_data", null, "REPEAT_STEP", "SHOW_MANUAL",
                             "MARK_PART", "CLOSE_WORK_ORDER", "order_part " };

        InteractionPolicy original = new InteractionPolicy();
        TallerPolicy taller = new TallerPolicy();

        System.out.printf("%-20s | %-22s | %-22s%n", "intent",
                "Original (conf/manos)", "TallerPolicy (conf/manos)");
        System.out.println("-".repeat(72));
        for (String intent : intents) {
            String orig;
            try {
                orig = original.requiresConfirmation(intent) + " / "
                     + original.canRunHandsFree(intent);
            } catch (NullPointerException e) {
                orig = "NullPointerException";
            }
            String nuevo = taller.requiresConfirmation(intent) + " / "
                         + taller.canRunHandsFree(intent);
            System.out.printf("%-20s | %-22s | %-22s%n",
                    intent == null ? "null" : "\"" + intent + "\"", orig, nuevo);
        }
    }
}
```

Resultado real de ejecutar TallerPolicyDemo:

| intent | InteractionPolicy (conf. / manos libres) | TallerPolicy (conf. / manos libres) |
|---|---|---|
| "NEXT_STEP" | false / true | false / true |
| "MAKE_PAYMENT" | true / false | true / false (desconocida en TallerGuía) |
| "ORDER_PART" | false / true | true / false |
| "UNKNOWN" | false / true | true / false |
| "delete_data" | false / true | true / false (desconocida) |
| null | NullPointerException | true / false (sin error) |
| "CLOSE_WORK_ORDER" | false / true | true / false |

### Respuesta e.4 · Intenciones que piden confirmación y las que van con manos libres

| Intención | Confirmación | Manos libres | Justificación |
|---|---|---|---|
| NEXT_STEP | No | Sí | Es reversible (se puede repetir o volver) y no tiene coste. Pedir confirmación en cada paso frenaría el trabajo. |
| REPEAT_STEP | No | Sí | Solo vuelve a mostrar o leer el paso actual; no tiene ningún efecto. |
| SHOW_MANUAL | No | Sí | Es solo lectura: no modifica datos ni genera costes. |

| Intención | Confirmación | Manos libres | Justificación |
|---|---|---|---|
| MARK_PART | No | Sí | Solo dibuja un resaltado sobre la imagen. Si se marca otra pieza, el mecánico lo ve de inmediato y lo corrige sin consecuencias. |
| ORDER_PART | Sí | No | Genera un pedido con coste para el taller. El ruido hace probable entender otra pieza, así que se confirma con una tarjeta (pieza, cantidad, coste) y un botón táctil grande. |
| CLOSE_WORK_ORDER | Sí | No | El vehículo pasa a «listo para entregar» y se avisa al cliente: efecto externo difícil de deshacer. Debe ser un acto deliberado, y conviene comprobar antes que todos los pasos están hechos. |

Criterio general: confirmación cuando la acción tiene coste o efecto externo irreversible; manos libres cuando es reversible, sin coste y sin efectos fuera de la pantalla. La confirmación debe hacerse por táctil (botón grande) y no repitiendo la voz, porque en un taller ruidoso un «sí» hablado se confunde con facilidad.

## Entrega

- Un documento con las respuestas de los apartados a) a e). Las respuestas deben estar razonadas: no basta con la solución, explica por qué.
- El código de los apartados d) y e) en archivos .java.
- Puedes usar la presentación y el PDF de la unidad. Cita la diapositiva o el concepto en el que te apoyas.